МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій
Кафедра системного проектування

ЗВІТ

про виконання лабораторної роботи №3
з курсу «Професійний Python»

«Ітератори, генератори та потокова обробка даних»

Виконав:
студент 3 курсу, групи ФЕП-31с
Гарбіч Данило

Перевірив:
професор
Свелеба Сергій Андрійович

Львів — 2026

2. Номер і тема лабораторної роботи

Лабораторна робота №3. «Ітератори, генератори та потокова обробка даних».

3. Мета роботи

Сформувати практичні навички роботи з ітераційною моделлю мови Python, протоколом ітератора (`__iter__` / `__next__`), генераторними функціями (`yield`, `yield from`), генераторними виразами (`generator expressions`), стандартним модулем `itertools`, концепцією відкладених лінивих обчислень (`lazy evaluation`) та організацією потокової обробки великих масивів даних без їхнього повного завантаження в оперативну пам'ять.

4. Номер індивідуального варіанта

Варіант №9: «Потокова обробка спортивних результатів».

5. Постановка задачі

Розробити потоковий конвеєр (`streaming lazy pipeline`) обробки великого набору даних для варіанта №9:

- Потокова валідація спортивних результатів (`streaming validation`);
- Фільтрація за видом спорту та категорією;
- Визначення найкращого результату без повної матеріалізації колекцій;
- Потоковий розрахунок середнього значення (`streaming average result`);
- Формування рейтингу результатів;
- Pairwise аналіз зміни послідовних результатів (`itertools.pairwise`);
- Пакетна обробка потоку (`batching`);
- Отримання перших N результатів за заданою умовою (`itertools.islice`);
- Експериментальне порівняння eager та lazy підходів на вибірках 10 000, 100 000 та 500 000 записів.

6. Опис input dataset

Вхідні дані зберігаються у форматі CSV із заголовками: `athlete,sport,result,category,timestamp`.

```
athlete,sport,result,category,timestamp
Ярослава Магучіх,Стрибки у висоту,2.10,Дорослі,2024-07-07T18:30:00
Ірина Геращенко,Стрибки у висоту,2.00,Дорослі,2024-07-07T18:25:00
Юлія Левченко,Стрибки у висоту,1.95,Дорослі,2024-07-07T18:15:00
```

7. Структура проєкту

```
lab3/
├── .gitignore                       # Ігнорування кешу, .venv та великих датасетів
├── pyproject.toml                   # Конфігурація пакета за стандартом PEP 621
├── README.md                        # Технічна інструкція користувача
├── REPORT.md                        # Повний текст звіту про виконання роботи
├── data/
│   └── input.csv                   # Зразок вхідного датасету
│
├── src/
│   └── stream_processor/           # Головний пакет потокової обробки
│       ├── __init__.py              # Версія програми (__version__ = "0.1.0")
│       ├── models.py               # NamedTuple SportsRecord, класи SportsRange/Iterator
│       ├── readers.py              # Генератори читання та очищення (yield from)
│       ├── parsers.py              # Потоковий CSV парсер
│       ├── filters.py              # Валідація та лінива фільтрація
│       ├── transformations.py      # Нормалізація та пакетування (batching)
│       ├── analytics.py            # Потокові агрегації (avg, min/max, pairwise)
│       ├── pipeline.py             # Збирання лінивого конвеєра
│       └── main.py                 # Демонстрація, вимірювання та бенчмарки
│
└── tests/
    └── test_stream_processor.py    # Модульні тести компонентів конвеєра
```

8. Пояснення iterable та iterator

Iterable — об'єкт, з якого можна отримати ітератор через метод `__iter__()` (наприклад, списки, файли, кортежі). Може ітеруватися багаторазово.

Iterator — об'єкт зі збереженням стану обходу, який має метод `__next__()`, повертає значення по одному та піднімає `StopIteration` після вичерпання. Є одноразовим (one-shot object).

9. Код власного iterator

```
class SportsRangeIterator:
    def __init__(self, start: int, stop: int) -> None:
        self.current = start
        self.stop = stop

    def __iter__(self) -> "SportsRangeIterator":
        return self

    def __next__(self) -> int:
        if self.current >= self.stop:
            raise StopIteration
        val = self.current
        self.current += 1
        return val

class SportsRange:
    def __init__(self, start: int, stop: int) -> None:
        self.start = start
        self.stop = stop

    def __iter__(self) -> SportsRangeIterator:
        return SportsRangeIterator(self.start, self.stop)
```

10. Приклад generator function

```
def clean_lines(lines: Iterator[str]) -> Iterator[str]:
    for line in lines:
        cleaned = line.strip()
        if cleaned:
            yield cleaned
```

11. Приклад yield

```
def filter_by_sport(records: Iterable[SportsRecord], sport: str) -> Iterator[SportsRecord]:
    target = sport.strip().lower()
    for record in records:
        if record.sport.strip().lower() == target:
            yield record
```

12. Приклад yield from

```
def read_lines(path: Path) -> Iterator[str]:
    with path.open("r", encoding="utf-8") as file:
        yield from file
```

13. Generator expression

```
high_jump_scores = (r.result for r in stream if r.sport == "Стрибки у висоту")
total_score = sum(high_jump_scores)
```

14. Використані засоби itertools

- `itertools.islice` — вибірка перших N записів без читання всього потоку.
- `itertools.pairwise` — аналіз динаміки зміни результатів між сусідніми спробами.
- `itertools.chain` — об'єднання декількох потоків в один послідовний ланцюг.

15. Схема pipeline для звіту

```
Input file (CSV)
      │
      ▼
read_lines() [yield from]
      │
      ▼
clean_lines()
      │
      ▼
parse_csv_rows() [DictReader, yield from]
      │
      ▼
validate_records() [validation, drop invalid]
      │
      ▼
normalize_athlete_names() [title-case]
      │
      ▼
filter_by_sport() / filter_by_category() [lazy filtering]
      │
      ▼
batched_records() [itertools.islice]
      │
      ▼
calculate_stream_statistics() / streaming_average() [aggregation]
      │
      ▼
Result (O(1) memory)
```

16. Код streaming reader

```
def read_lines(path: Path) -> Iterator[str]:
    with path.open("r", encoding="utf-8") as file:
        yield from file
```

17. Validation stage

```
def validate_records(rows: Iterable[dict[str, str]], invalid_counter: list[int] | None = None)
    for row in rows:
        try:
            athlete = row.get("athlete", "").strip()
            sport = row.get("sport", "").strip()
            category = row.get("category", "").strip()
            timestamp = row.get("timestamp", "").strip()
            result_val = float(row.get("result", ""))

            if not athlete or not sport or not category or result_val < 0:
                if invalid_counter is not None:
                    invalid_counter[0] += 1
                continue

            yield SportsRecord(athlete, sport, result_val, category, timestamp)
        except (ValueError, TypeError, KeyError):
            if invalid_counter is not None:
                invalid_counter[0] += 1
            continue
```

18. Filtering stage

```
def filter_by_sport(records: Iterable[SportsRecord], sport: str) -> Iterator[SportsRecord]:
    target = sport.strip().lower()
    for record in records:
        if record.sport.strip().lower() == target:
            yield record
```

19. Transformation stage

```
def normalize_athlete_names(records: Iterable[SportsRecord]) -> Iterator[SportsRecord]:
    for record in records:
        yield record._replace(athlete=record.athlete.title())
```

20. Aggregation stage

```
def streaming_average(records: Iterable[SportsRecord]) -> float:
    total = 0.0
    count = 0
    for r in records:
        total += r.result
        count += 1
    return round(total / count, 2) if count else 0.0
```

21. Batching

```
def batched_records(records: Iterable[SportsRecord], batch_size: int) -> Iterator[list[SportsRe
    iterator = iter(records)
    while True:
        batch = list(islice(iterator, batch_size))
        if not batch:
            return
        yield batch
```

22. Eager implementation

```
def eager_read_all(path: Path) -> list[dict[str, str]]:
    with path.open("r", encoding="utf-8", newline="") as file:
        reader = csv.DictReader(file)
        return list(reader)  # Повна матеріалізація в пам'яті
```

23. Lazy implementation

```
def lazy_processing(path: Path, sport: str) -> float:
    stream = build_pipeline(path, sport_filter=sport)
    return streaming_average(stream)  # Одночасно в пам'яті лише один запис
```

24. Таблиця elapsed time

Кількість записів	Eager час виконання	Lazy час виконання	Time-to-first-result (Lazy)
10 000	0.1351 с	0.2183 с	0.000215 с
100 000	1.1520 с	2.0398 с	0.002283 с
500 000	5.9050 с	9.6891 с	0.000302 с

25. Таблиця peak memory

Кількість записів	Eager пікова пам'ять	Lazy пікова пам'ять	Коефіцієнт економії RAM
10 000	4.88 МБ	0.22 МБ	**22.2**×
100 000	48.65 МБ	0.22 МБ	**221.1**×
500 000	243.90 МБ	0.07 МБ	**3484.3**×

26. Аналіз результатів

1. **Використання пам'яті:** у eager-підході обсяг пам'яті строго масштабується лінійно від розміру вхідного файлу (досягає 243.90 МБ на 500k рядках). Lazy-підхід демонструє стабільні показники (0.07–0.22 МБ), надаючи економію пам'яті понад **3484 рази**.

2. **Загальний час виконання:** Eager-підхід завершується дещо швидше на повній вибірці через відсутність накладних витрат генераторних викликів. Отже, генератори оптимізують насамперед пам'ять, а не час повного прогону.

3. **Time-to-first-result:** Lazy-конвеєр повертає перші валідні дані практично миттєво (за 0.0003 с), тоді як Eager-варіант блокує виконання на 5.9 секунд.

27. Відповіді на контрольні запитання

1. Що таке iterable?

Об'єкт у Python, з якого можна отримати ітератор за допомогою виклику iter(); реалізує метод __iter__() або __getitem__().

2. Що таке iterator?

Об'єкт, що зберігає стан обходу послідовності, реалізує метод __next__() та метод __iter__(), що повертає self.

3. Чим iterable відрізняється від iterator?

Iterable — це багаторазове джерело даних. Iterator — це одноразовий об'єкт-покажчик, що вичерпується після завершення обходу.

4. Для чого використовується iter()?

Для явного отримання об'єкта-ітератора з довільного ітерованого об'єкта.

5. Для чого використовується next()?

Для запиту наступного елемента в ітератора або генерації винятку StopIteration, якщо елементи вичерпано.

6. Що робить __iter__()?

В iterable створює новий ітератор; в ітераторі повертає сам об'єкт self.

7. Що робить __next__()?

Повертає наступне значення послідовності або піднімає виняток StopIteration.

8. Що таке StopIteration?

Стандартний виняток Python, що сигналізує про завершення ітерації.

9. Як цикл for використовує iterator protocol?

Викликає iter() на початку, в циклі викликає next() та автоматично обробляє виняток StopIteration.

10. Чому iterator часто називають one-shot object?

Тому що він рухається лише вперед і після повного проходу не може бути скинутий на початок.

11. Чому iter(iterator) is iterator?

Оскільки метод __iter__() ітератора зобов'язаний повертати сам об'єкт self для сумісності з протоколом.

12. Чи можна повторно використати exhausted generator?

Ні, вичерпаний генератор порожній; для повторного обходу треба створити новий екземпляр генератора.

13. Що таке generator function?

Функція, яка містить ключове слово yield і повертає об'єкт-генератор замість прямого виконання тіла.

14. Яку роль виконує yield?

Повертає поточне значення, заморожує виконання функції та зберігає стан локальних змінних.

15. Чим yield відрізняється від return?

return остаточно завершує роботу функції; yield призупиняє функцію з можливістю продовження з тієї ж точки.

16. Коли фактично починає виконуватися generator function?

Під час першого виклику next() над отриманим об'єктом-генератором.

17. Що зберігає generator між викликами next()?

Покажчик поточної інструкції, локальні змінні, фрейм стека та стан обробників винятків.

18. Що таке generator expression?

Компактна синтаксична форма (expr for item in iterable), яка створює лінивий генератор без виділення пам'яті під список.

19. Чим generator expression відрізняється від list comprehension?

List comprehension матеріалізує весь список у пам'яті одразу, а generator expression генерує елементи по одному за вимогою.

20. Що таке lazy evaluation?

Обчислення значень за запитом без попереднього розрахунку всього масиву.

21. Що таке eager evaluation?

Жадібне обчислення, коли весь результат рахується й зберігається в пам'яті відразу.

22. Які переваги lazy evaluation?

Мінімальне споживання пам'яті, миттєвий доступ до перших значень, підтримка нескінченних потоків та раннього виходу (early termination).

23. Які недоліки lazy evaluation?

Одноразовість обходу, неможливість прямої індексації або швидкого визначення довжини len().

24. Що таке streaming?

Потокова обробка даних порціями без повного завантаження датасету в оперативну пам'ять.

25. Чому streaming важливий для великих files?

Він унеможливлює вичерпання RAM (MemoryError), якщо розмір файлу перевищує обсяг доступної оперативної пам'яті.

26. Чому read() може бути проблемою для multi-GB file?

Оскільки метод read() зчитує весь багатогігабайтний файл в єдиний монолітний рядок у пам'яті.

27. Чому file object є iterable?

Він реалізує протокол ітератора, ліниво повертаючи рядки через буфер по одному за викликом next().

28. Для чого використовується yield from?

Для прозорого делегування генерації елементів іншому генератору чи ітерованому об'єкту.

29. Що таке itertools?

Стандартний високоефективний модуль Python для комбінаторних і потокових операцій над ітераторами.

30. Для чого itertools.count()?

Генерує нескінченну послідовність чисел із заданим кроком: count(start, step).

31. Чому count() потрібно обмежувати?

Оскільки без обмеження (наприклад, через islice) операція перетворення на список ніколи не завершиться.

32. Для чого islice()?

Для виконання зрізу по ітератору без матеріалізації всієї послідовності.

33. Для чого chain()?

Для послідовного склеювання декількох ітераторів в один суцільний потік.

34. Як працює takewhile()?

Повертає елементи, доки предикат повертає True, і негайно зупиняється на першому False.

35. Як працює dropwhile()?

Пропускає елементи, доки предикат повертає True, після чого повертає всі наступні елементи до кінця.

36. Для чого accumulate()?

Для обчислення накопичувальних значень (кумулятивної суми тощо).

37. Для чого pairwise()?

Для попарного ковзного аналізу сусідніх елементів: (s0, s1), (s1, s2)...

38. Як працює groupby()?

Групує поспіль розташовані елементи з однаковим ключем групування.

39. Чому перед groupby() часто потрібне sorting?

Тому що groupby не групує елементи з однаковими ключами, якщо вони розділені іншими ключами в потоці.

40. Що робить product()?

Обчислює декартовий добуток вхідних послідовностей.

41. Чим combinations() відрізняється від permutations()?

combinations повертає невпорядковані комбінації без повторів, а permutations враховує порядок елементів.

42. Чому combinatorial iterators можуть бути небезпечними навіть при lazy execution?

Кількість їхніх комбінацій зростає експоненційно або факторіально, що викличе колосальний час виконання.

43. Що таке batching?

Розбиття безперервного потоку даних на порції фіксованого розміру для масової обробки.

44. Які переваги batching?

Оптимізація операцій вводу/виводу, роботи з базою даних чи зовнішніми API.

45. Що таке lazy pipeline?

Ланцюжок генераторів, де кожен етап обробляє елементи по одному в міру потреби наступного етапу.

46. Які stages може містити pipeline?

Джерело (source), парсер, валідатор, фільтр, трансформатор, пакетувальник, агрегатор (sink).

47. Що є source у streaming pipeline?

Первинне джерело потоку (файл, мережевий сокет, нескінченний генератор).

48. Що є transformation?

Проміжний генератор, який змінює формат чи поля кожного окремого запису.

49. Що є sink?

Кінцевий споживач потоку, який матеріалізує дані, записує їх або рахує агрегати (sum, count).

50. Чи гарантує generator менший total execution time?

Ні, часто генератор трохи повільніший через накладні витрати на виклики next() та перемикання фреймів стека.

51. Яка основна memory advantage generator?

Константна пікова пам'ять O(1), незалежна від загального обсягу оброблюваного датасету.

52. Що таке time to first result?

Час від запуску програми до отримання найпершого обробленого елемента споживачем.

53. Чому lazy pipeline добре підтримує early termination?

Тому що обчислення можна перервати в будь-який момент, не витрачаючи час на читання решти файлу.

54. Чому exception у generator може виникати із затримкою?

Оскільки код генератора не виконується доти, доки споживач не дійде до рядка з помилкою через виклик next().

55. Чому list(generator) може бути проблемою?

Це матеріалізує весь потік у списку, повністю нівелюючи перевагу генератора в економії оперативної пам'яті.

56. Що трапиться при list(count())?

Нескінченний цикл, зависання програми та падіння через переповнення пам'яті (MemoryError).

57. Як generator може впливати на lifecycle file resource?

Якщо генератор не вичерпано або не закрито явно, контекстний менеджер with залишає файл відкритим довше, ніж очікувалося.

58. Як тестувати невеликий generator?

Перетворити його на список list(gen) і порівняти з очікуваними даними через assert.

59. Як тестувати infinite generator?

За допомогою itertools.islice(gen, N) обмежити кількість елементів фіксованим числом N.

60. Коли краще використовувати list, а коли generator?

List кращий для багаторазового використання, сортування та індексації невеликих обсягів даних. Generator незамінний для великих файлів, потоків та економії пам'яті.

28. Висновок

Під час виконання лабораторної роботи №3 було успішно опановано ітераційну модель Python, принципи відкладених обчислень та побудову високоефективних потокових конвеєрів:

1. **Ітератори та протокол:** створено власний клас ітератора `SportsRangeIterator` та контейнера `SportsRange`, що закріпило розуміння взаємодії `__iter__()`, `__next__()` та винятку `StopIteration`.
2. **Генераторний конвеєр:** розроблено лінивий конвеєр обробки спортивних результатів за варіантом №9, що включає фази читання (`read_lines`), парсингу, валідації (`validate_records`), лінивої фільтрації, нормалізації та пакетування (`batching`).
3. **Модуль itertools:** ефективно застосовано інструменти `islice` (зріз без матеріалізації), `pairwise` (попарний аналіз різниць результатів) та `chain` (об'єднання потоків).
4. **Порівняльний аналіз пам'яті та часу:** практичні вимірювання на вибірках до 500 000 записів продемонстрували колосальну перевагу lazy-підходу за піковою пам'яттю (економія у **3484 рази** на 500k рядках: 0.07 МБ проти 243.9 МБ). Водночас підтверджено, що lazy-підхід забезпечує практично миттєве повернення першого результату (0.0003 с проти 5.9 с).

Отримані знання дозволяють будувати промислові масштабовані системи аналізу даних, що працюють з великими файлами в умовах обмежених ресурсів оперативної пам'яті.